AsymMark: Weakly Asymmetric Behavioral Watermarking for LLM Agents

Abstract—**Autonomous LLM agents act through tool calls, subgoal choices, and embodied decisions, so provenance must cover behavior trajectories rather than final text alone. Distribution-preserving behavioral watermarks are the deployable class, since they embed provenance without distorting the agent's policy. However, the leading construction is symmetric: it requires verifiers to replay the exact per-step probabilities used at embedding. Realistic audits rarely meet this bar, because verifiers face model drift between embedding and verification, closed or quantized inference endpoints, and APIs that expose only a truncated top-k view. Under these constraints, the most a verifier can reliably reconstruct is the rank order of plausible behaviors at each step, not their calibrated probabilities, and symmetric probability-bin decoders structurally collapse on this residual channel because their bins depend on probability mass rather than rank. We propose AsymMark, a weakly asymmetric framework that decouples the policy-preserving embedding channel from a verifier-reconstructible decoding invariant, and instantiate AsymMark-R using top-k rank order as that invariant, a channel more stable than calibrated probabilities and more informative than the selected action alone. We further introduce a metric that separates the nominal embedding opportunity from the capacity that actually survives the verifier's view, exposing how much provenance a scheme retains under realistic audits. On ALFWorld and ToolBench across DeepSeek and Gemini Flash, AsymMark-R preserves task utility and behavior-frequency stealth, retains provenance under rank-only verification, and reliably recovers multi-bit payloads aggregated across audit windows. Behavioral provenance should therefore be designed around the channel the verifier can actually reconstruct, decoding from the invariant that survives rather than from the probabilities that do not.**

1. Introduction

Large language model (LLM) systems are rapidly shifting from passive text generators to agents that perceive context, plan, call tools, and act across multiple steps [1], [2], [3]. This shift changes what provenance must mean. For a chatbot, attributing the final string may be enough. For an agent, the consequential object is often the trajectory: which tool it selected, which subgoal it pursued, which API it invoked, or which embodied action it chose. These planning behaviors can affect safety, accountability, intellectual-property protection, and post-hoc auditing even when the final natural-language output is short or absent.

Content watermarking for LLM outputs has made substantial progress [4], [5], but it does not directly solve behavioral provenance. Planning behaviors are not token-native: an LLM's reasoning text may be compiled into structured tool calls or environment actions, discarding the lexical signal used by token watermarks. In response, AgentMark [6] formulated behavioral watermarking as distribution-preserving sampling over an explicit per-step behavior distribution. Its concrete construction, which we call AgentMark-F, embeds multi-bit identifiers while preserving the marginal behavior distribution, and it tolerates partial logs through erasure coding.

However, AgentMark-F is symmetric: decoding requires the verifier to reconstruct the same probability distribution used by the embedder. This is a strong assumption in real deployments. The verifier may access a rounded top-k API, a model after provider-side updates, a different serving stack, or a proxy model used for audit. These changes often preserve the relative order of plausible behaviors while shifting the numeric probabilities.

This is not only an implementation inconvenience; it is a structural verifier-channel failure mode. AgentMark-F's differential bins are functions of probability masses. Two distributions can induce the same top-k order but move a cumulative boundary across a rank position, causing the same selected behavior to decode to a different bin. Thus a state-of-the-art exact-channel behavioral watermark can look strong under its native metric yet fail in the practically important rank-only audit channel. The lesson is broader than one implementation: behavioral watermarking should measure the channel available to the verifier, not only the capacity available to the embedder.

This paper asks whether behavioral provenance can be robust when the verifier has partial channel knowledge rather than calibrated probabilities. We formulate AsymMark, a weakly asymmetric behavioral watermarking framework for agent trajectories. In this framework, the embedder may use the full behavior distribution P_t to preserve the agent policy, while the verifier decodes from an invariant that can be reconstructed from its restricted view. This setting is weaker than exact-probability verification but stronger than fully asymmetric verification with no channel knowledge. The rank-order view is the first concrete instance: common audit interfaces often expose candidate rankings without calibrated probabilities, and rank is more stable than calibrated probability values while still more informative than the selected action alone.

We instantiate the framework as AsymMark-R, a top-k rank-based behavioral watermark. The rank-splitting primitive is not claimed as a new steganographic primitive; it is adapted from weakly asymmetric rank-based steganography [7]. Our contribution is to make the weak-asymmetry abstraction explicit for AgentMark's behavior channel, where candidate actions rather than text tokens carry provenance, and to specify the synchronization, coding, and audit protocol needed for logged agent trajectories. The encoder sorts behaviors by probability and recursively partitions the top-k list into even and odd ranks. At each level, a binary distribution-preserving primitive uses probability masses to choose a group without changing the marginal policy. The decoder, by contrast, never uses probability values: the selected behavior's rank determines the path through the same interleaved partition tree, and odd branches reveal embedded bits. A fixed-budget deterministic random bit generator (DRBG) schedule ensures that encoder and decoder consume the same pseudorandomness even when no bit is embedded at a level.

We organize the evaluation around effective capacity,

$$C_{\text{eff}} = C_{\text{nom}} \times \text{DSR}, \tag{1}$$

where C_{nom} is the nominal number of recoverable coded bits exposed by the channel and DSR is the decode success rate under the verifier's view. This yardstick separates a watermark that has high capacity only under exact probabilities from one that yields lower nominal capacity but survives a degraded verifier channel. It also makes packet scarcity explicit: individual traces contribute packets, and attribution is evaluated over pooled audit windows once enough independent equations accumulate.

The claim is intentionally scoped. The paper establishes a channel-knowledge tradeoff: top-k ranks can preserve decoder signal when exact probabilities are unavailable. It does not claim attribution from selected actions alone; audit-grade attribution still requires a pre-registered key/payload claim, calibrated rank reconstruction, and end-to-end RLNC recovery under the deployment logging stack.

Our contributions are:

- **Weakly asymmetric behavioral watermarking framework.** We formulate AsymMark as a framework that decouples the policy-preserving embedding channel from a verifier-reconstructible decoding invariant. The invariant must be recoverable from the verifier's restricted view, independent of calibrated probability values, and sufficient to index the decoder's partition path. This framing makes verifier-channel knowledge an explicit design object rather than an implicit replay assumption.
- **Rank instantiation and audit protocol.** We instantiate AsymMark as AsymMark-R, using top-k rank order as the decoding invariant. The rank primitive is adapted from weakly asymmetric steganography [7]; the new contribution is its agent-behavior instantiation with fixed $\lceil \log_2 n_t \rceil$ DRBG synchronization and deterministic RLNC over logged trajectories.
- **Verifier-channel failure mode, formalized and measured.** We identify probability-bin instability as a distinct robustness dimension, formalize it in Lemma 1, and empirically quantify it: under rank-stable top-k truncation, AgentMark-F's bins still change on 13.7% of ALFWorld DeepSeek steps at top-10 and 56.8% at top-4, with matching decoded-bit flips of 11.1% and 51.3%. We also measure rank-noise path corruption against the Proposition 2 empirical envelope in Fig. 6.
- **Effective-capacity and audit-window evidence model.** We adopt $C_{\text{eff}} = C_{\text{nom}} \times \text{DSR}$ to score capacity that survives the verifier's channel, and model payload evidence as cumulative packet recovery over pooled audit windows through the packet-scarcity bound in Proposition 3.
- **Four-axis evaluation with live cross-model calibration.** We evaluate on ALFWorld [8] and ToolBench [9] across DeepSeek and Gemini Flash along capacity, stealth, robustness, and evidence-strength axes. Beyond offline channel sweeps, the live ToolBench cross-model rerank study exposes a direction-dependent operating point: DeepSeek-to-Gemini is best at top-3, while Gemini-to-DeepSeek is best at top-5 (Fig. 8).

The rest of the paper proceeds as follows. Section 2 motivates behavioral watermarking and weak asymmetry. Section 3 defines the agent behavior channel, adversary, and verifier knowledge. Section 4 presents the rank encoder, decoder, synchronization, and coding layer. Sections 5 and 6 analyze security and evaluate utility, stealth, and rank-only recovery. Section 7 discusses operational limits, Section 8 covers related work, Section 9 covers validity and ethics, and Section 10 concludes. The appendices provide the optional depth behind the main claims: distribution preservation, DRBG synchronization, RLNC recovery, rank stability, and artifact mapping.

2. Background and Motivation

2.1. Behavioral Watermarking for Agents

An LLM agent operates over a trajectory of observations, planning behaviors, and execution actions. At each step t, the agent chooses a behavior b_t from a finite admissible set $\mathcal{B}_t$. Examples include ALFWorld navigation actions, ToolBench API calls, or social-simulation behaviors in OASIS [10]. AgentMark models this choice as sampling from an elicited behavior distribution P_t and replaces ordinary sampling with a distribution-preserving encoder:

$$\hat{b}_t \leftarrow \mathsf{Enc}(P_t, r_t), \quad \Pr[\hat{b}_t = b] = P_t(b). \tag{2}$$

The distribution-preservation requirement is central. Agent trajectories are long-horizon and stateful; even small per-step distribution shifts can compound into task drift, failures, or visibly abnormal behavior.

2.2. The Symmetry Bottleneck

The concrete AgentMark-F construction uses differential recombination over P_t followed by cyclic-shift encoding. This is efficient when both embedder and verifier share exact probabilities. The same property makes it brittle under probability perturbation: if the verifier's reconstructed P'_t

differs from P_t, recombined bins may differ, so the selected behavior is interpreted under the wrong bin structure.

The weakness is not merely numerical. Many verification interfaces intentionally withhold calibrated probabilities. Auditors may receive ranked candidates, top-k choices, or sampled logs. Even when probabilities are available, temperature scaling, quantization, floating-point differences, model upgrades, and fine-tuning can perturb values. These transformations frequently preserve a useful invariant: the order of high-probability behaviors.

This observation is especially natural for agents. Candidate behaviors are usually semantic alternatives produced by a planning prompt: a tool call, a navigation operation, a search/refine step, or a social action. A downstream auditor may be able to ask a model to rank plausible next behaviors from the same logged context even if the exact sampling distribution is hidden. In this sense, behavior-level verification has a useful middle ground that token-level watermarking often lacks: ranks are easier to reproduce than calibrated probabilities, and more informative than the selected action alone.

This gives a three-level hierarchy. AgentMark is the symmetric behavioral-watermarking framework, and AgentMark-F is its full-probability instance. AsymMark is the weakly asymmetric counterpart: it keeps the same behavior-trajectory carrier and preserves the behavior policy P_t, but allows the decoder to depend on a verifier-reconstructible invariant rather than exact probability replay. AsymMark-R is the rank-order instance of this framework, using top-k rank σ_t as the verifier channel rather than claiming the rank primitive itself is new.

2.3. Weak Asymmetry

Provably secure steganography traditionally separates symmetric settings, where sender and receiver share channel distributions, from asymmetric settings, where the receiver has no channel oracle [11], [12]. Weak asymmetry occupies the middle ground: the sender has the full channel, while the receiver has partial but nontrivial channel knowledge [7]. We lift this idea from a single rank primitive to an agent-watermarking framework. The embedder uses P_t to preserve the agent policy; the verifier uses a lower-dimensional invariant that is easier to reconstruct than calibrated probabilities. In AsymMark-R, that invariant is $\sigma_t = \operatorname{argsort}_k(P_t)$.

3. Problem Formulation

3.1. Agent Behavior Channel

For a trajectory of length T, each step t has a behavior set $\mathcal{B}_t = \{b_{t,1}, \ldots, b_{t,N_t}\}$ and a behavior distribution P_t induced by the agent and environment context. A watermarking key K_{sh} and step context h_t derive per-step pseudorandomness. The embedder encodes a payload $m \in \{0,1\}^L$ into selected behaviors $\hat{b}_1, \ldots, \hat{b}_T$ while preserving P_t at each step.

The verifier observes a possibly incomplete set of indices $I \subseteq \{1, \ldots, T\}$ and, for each observed step, the selected behavior $\hat{b}_t$ plus channel side information. In the symmetric setting, the side information is P_t or an exact reconstruction. In the weakly asymmetric setting, the side information is a verifier-reconstructible invariant; in AsymMark-R, it is the top-k ordering σ_t.

Verifier knowledge. We distinguish three verifier views. In the *exact view*, the verifier receives P_t and can run the same decoder as the embedder. In the *rank view*, the verifier receives only $\sigma_t^k = (b_{(1)}, \ldots, b_{(k)})$, the ordered list of the top-k behaviors. In the *selected-only view*, the verifier receives only $\hat{b}_t$ and no usable channel side information. The exact view is a useful upper bound; the rank view is the deployment target of AsymMark-R; the selected-only view is a negative control showing that behavioral provenance cannot be recovered from actions alone without some channel side information.

For evaluation we instantiate these views as a channel-knowledge ladder. $L0$ is exact probability access. $L1$–$L5$ progressively restrict the verifier to top-10, top-8, top-6, top-4, and top-2 ordered candidates. $L6$ is the selected-only negative control. This ladder is not a claim that every API exposes exactly these levels; it is a controlled way to ask how usable capacity changes as the verifier loses calibrated probability information.

Decodable invariants. AsymMark separates the embedding channel from the decoding channel through a per-step invariant $z_t = \phi(P_t, \mathcal{B}_t)$. A useful weak-asymmetric invariant must satisfy three conditions. First, it must be reconstructible from the verifier's restricted view, such as a rounded top-k API or proxy scorer. Second, it must be independent of the numeric probability values used by the embedder to preserve $\Pr[\hat{b}_t = b] = P_t(b)$. Third, it must be sufficient to index the shared partition path that turns a selected behavior into coded bits. Top-k rank order satisfies these conditions for AsymMark-R: the verifier can reconstruct an ordered candidate list, probability-value perturbations do not change the path when the order is stable, and the selected behavior's rank identifies the even/odd partition decisions. Other invariants are possible in principle; Section 7 returns to this broader design space.

Rank consistency. For two distributions P_t and P_t' over the same candidate list, define top-k rank consistency as

$$\operatorname{argsort}_k(P_t) = \operatorname{argsort}_k(P_t'). \qquad (3)$$

This condition permits arbitrary probability-value changes as long as the verifier preserves the ordered top-k set. It covers monotone transformations such as temperature rescaling, but it also covers less structured API changes whose empirical effect is to leave ranks unchanged. When the equality fails, we measure degradation through rank-noise experiments rather than assuming correctness.

3.2. Adversary and Requirements

We consider three perturbations. First, *step erasure*: some logged steps are missing due to execution failures, platform deletion, or incomplete telemetry. Second, *distribution perturbation*: a verifier reconstructs P'_t whose numeric values differ from P_t. Third, *rank noise*: the verifier's top-k order disagrees with the embedder's order, modeling top-k instability or proxy-model disagreement.

The adversary observes watermarked trajectories and may know the algorithm and public parameters, including k, but not K_{sh}. This is the standard secret-key provenance setting: a platform or model owner embeds an identifier, while a later verifier checks a claimed key and payload. We do not rely on security through obscurity of the rank partition or coding scheme.

The adversary has two goals. A *removal* adversary tries to make a genuinely watermarked trajectory verify as inconclusive or negative by deleting steps, perturbing ranks, or changing behavior serialization. A *framing* adversary tries to make an unwatermarked or differently watermarked trajectory verify under another party's claim. Claim binding and false-positive thresholds address framing when the key is secret; arbitrary trajectory rewriting, compromised keys, and compromised logging infrastructure are outside the cryptographic claim and are handled as deployment assumptions.

The audit boundary is therefore explicit. The verifier needs a trajectory log whose step contexts and selected behaviors are bound by the embedding service or another trusted logging layer, plus a deterministic mapping from logged behaviors to verifier-side candidates. The watermark can tolerate missing steps and imperfect rank reconstruction, but it cannot distinguish a genuine erasure from a log rewritten by an untrusted recorder. We treat log integrity and candidate canonicalization as deployment prerequisites and evaluate the watermark channel conditional on those prerequisites.

A scheme should satisfy:

- **Utility preservation**: $\Pr[\hat{b}_t = b] = P_t(b)$ per step.
- **Weak-asymmetric correctness**: if the verifier reconstructs the same invariant z_t as the embedder, decoding returns the same bits; for AsymMark-R, this means $\mathrm{argsort}_k(P'_t) = \mathrm{argsort}_k(P_t)$.
- **Low false positives**: for a fixed pre-registered L-bit claim, a clean or unrelated trajectory should recover that exact payload with probability 2^{-L} when decoded bits are independent of the claim.
- **Erasure tolerance**: payload recovery should degrade gracefully when only a subset of steps is observed.

Detection statistic. For an audit window, the verifier decodes a multiset of coded bits and attempts to recover payload $\hat{m}$. A positive attribution requires $\hat{m} = m$ for a pre-registered claimed payload after packets have been accumulated over the window. A failed recovery is treated as negative only when the verifier has enough rank-consistent evidence and independent equations; otherwise it is inconclusive. Payload recovery is therefore the provenance claim: before the pooled window has enough independent equations for full RLNC recovery, the audit should keep collecting evidence or report an inconclusive outcome.

3.3. Non-Goals

AsymMark-R is not a general-purpose watermark removal defense. If an adversary controls the agent and can arbitrarily rewrite the trajectory, no behavioral watermark can force attribution without an external logging root of trust. Our target is post-hoc verification for trajectories that are generated by an honest embedder but later audited through an imperfect channel interface. We also do not assume the verifier can re-run the exact original model. The verifier may use a same-family model, a refreshed endpoint, or a restricted API as long as the top-k order remains sufficiently stable.

4. Design

4.1. Overview

AsymMark-R, the rank instance of AsymMark, has four components, summarized in Fig. 1. First, the agent elicits or estimates P_t over candidate behaviors, following AgentMark's black-box-compatible behavior-distribution interface [6]. Second, the encoder computes the rank invariant by sorting behaviors and restricting embedding to the top-k region. Third, it applies the interleaved rank split from weakly asymmetric steganography [7] to behavior candidates rather than text symbols, preserving the behavior distribution while making the selected rank decodable. Fourth, deterministic RLNC maps the payload into a stream of coded bits so that the verifier can solve for the payload after observing enough independent coded bits.

4.2. Binary Distribution-Preserving Primitive

At a tree node, the encoder must choose between two groups with masses S_0 and S_1 while optionally embedding one bit. Assume $S_0 \geq S_1$ and $S = S_0 + S_1$. Given message bit $m \in \{0, 1\}$ and $r \leftarrow U[0, 1)$ from the DRBG, define

$$r_m = (rS + \tfrac{1}{2}Sm) \bmod S. \tag{4}$$

BinEnc selects group 0 if $r_m < S_0$ and group 1 otherwise. If group 1 is selected, one bit is embedded; if group 0 is selected, no bit is consumed and the same bit is retried at the next level. Because r_m is uniform over $[0, S)$ for uniform r, the selected group has exactly the desired marginal probabilities S_0/S and S_1/S.

Why zero-bit branches are intentional. The primitive gives up a bit when the high-mass group is selected because forcing every branch to carry a bit would require reweighting the group probabilities. The deferred-bit design is the same kind of tradeoff used by distribution-preserving steganography: capacity is opportunistic, but the cover distribution is unchanged. In agent settings this is a feature rather than a cosmetic constraint, because utility regressions caused by shifted planning choices are easier for users and downstream systems to notice than small variations in textual style.

Embedder: full channel	→	Trajectory log	→	Verifier: partial channel
Agent context h_t and candidate behaviors $\mathcal{B}_t$. Elicit probabilities P_t and sort top-k order σ_t. Use P_t to sample a watermarked behavior with the same marginal distribution.		Logged behavior $\hat{b}_t$, step context, claimed payload identifier, and audit metadata. The log may omit steps or be checked after model/API changes.		Reconstruct only the top-k order σ'_t from a restricted API or proxy model. Decode bits from the rank of $\hat{b}_t$; solve RLNC equations for the payload.

Figure 1. System view of AsymMark-R, the rank-order instance of AsymMark. The embedder needs calibrated probabilities to preserve utility; the verifier needs only the rank invariant for decoding, plus enough observed coded bits for RLNC recovery.

4.3. Rank Encoder

Let σ_t be the descending rank ordering of P_t. At step t, let $n_t = \min(k, |\mathcal{B}_t|)$ be the actual number of visible candidates in the top-k view. The encoder first samples whether to enter the visible region with probability $S_{\text{top}} = \sum_{i=1}^{n_t} P_t(\sigma_t(i))$. If not, it samples from the tail proportionally and embeds no bits. Inside the visible region, it normalizes probabilities and applies recursive splitting:

1) Split the current sorted list into even ranks $(0, 2, 4, \ldots)$ and odd ranks $(1, 3, 5, \ldots)$.
2) Use BinEnc on the two group masses.
3) Recurse into the selected group until a single behavior remains.

Interleaving is important. A top-half/bottom-half split would place most mass in the first group, producing low embedding opportunity. Even/odd splitting balances mass while preserving a rank path that the verifier can recover.

4.4. Rank Decoder

Given selected behavior $\hat{b}_t$ and top-k order σ_t, the decoder sets $n_t = \min(k, |\sigma_t|)$ and finds the zero-indexed rank q of $\hat{b}_t$ within the visible prefix. If $\hat{b}_t$ is outside that prefix, the step yields no bits. Otherwise, the decoder reconstructs the path by repeatedly reading the parity of q:

$$q_0 = q, \quad q_{\ell+1} = \lfloor q_\ell / 2 \rfloor. \tag{5}$$

Whenever q_ℓ is odd, the selected path entered the odd group and one bit was embedded. The value of that bit is determined by the same DRBG value used at level ℓ: the implementation decodes it as 1 if $r_\ell < 0.5$ and 0 otherwise. The decoder consumes exactly $\lceil \log_2 n_t \rceil$ DRBG values per step, matching the encoder's fixed synchronization budget. The common shorthand $\lceil \log_2 k \rceil$ is the saturated-candidate upper bound when $n_t = k$. Figure 2 summarizes the one-step encoder and decoder side by side.

4.5. A Second Invariant Example

AsymMark is not tied to rank order as its only possible invariant. As a minimal second instance, consider an unordered-set keyed partition channel. The verifier observes the selected behavior $\hat{b}_t$ and the unordered top-k candidate set S_t, but not the order induced by P_t. The embedder and verifier derive a canonical order

$$\pi_t = \text{sort}_{a \in S_t}\big(\text{HMAC}(K_{\text{sh}}, h_t, a)\big) \tag{6}$$

using the shared key, logged context, and action identifier. The encoder then applies the same binary primitive to the canonical sequence π_t rather than to the probability rank order σ_t; the verifier reconstructs π_t from set membership alone and decodes from the position of $\hat{b}_t$ in that keyed order. The invariant is therefore unordered set membership plus keyed canonicalization, not calibrated probabilities or rank order.

This instance is mainly a constructive witness for the framework definition. On balanced candidate distributions, keyed partitions can preserve the source distribution while remaining invariant to arbitrary permutations of the verifier's set view. We do not use this unordered-set instance as the main evaluated system: for skewed real behavior distributions, additional balancing would be needed to preserve both capacity and distribution fidelity. The rank instance remains the evaluated operating point because it gives a stronger and naturally exposed invariant for ALFWorld and ToolBench.

4.6. Coding Layer

Per-step embedding capacity varies with P_t, k, and trajectory length. To avoid requiring a contiguous bitstream, AsymMark-R uses deterministic RLNC over GF(2), following the standard random linear network coding principle [13]. For payload $m \in \{0, 1\}^L$, the i-th coded bit is

$$c_i = \langle a_i, m \rangle \bmod 2, \tag{7}$$

where coefficient vector a_i is generated deterministically from a stream key and index i. The verifier collects decoded coded bits and solves the resulting linear system by Gaussian elimination over GF(2). This layer is inherited from AgentMark's erasure-resilience design but is independent of whether the step decoder is symmetric or rank-only.

The coding layer also cleanly separates two questions that are often conflated. The step decoder asks whether an observed behavior can yield a reliable coded bit under the verifier's channel view. RLNC asks whether enough independent coded bits have survived to solve for the payload. This separation is useful for weak asymmetry: a rank-only decoder may produce fewer bits than an exact-probability decoder, but the coding layer can still recover when the

Encoder: uses probabilities to preserve policy	Decoder: uses ranks, never probabilities
Input: P_t, context h_t, key $K_{\rm sh}$, coded stream M, top-k. 1. Derive $\mathcal{G}_t$ from $(K_{\rm sh}, h_t)$. 2. Sort behaviors: $\sigma_t \leftarrow \mathrm{argsort}(P_t)$; set $n_t = \min(k, \vert\mathcal{B}_t\vert)$. 3. Sample tail with probability $1 - \sum_{i \le n_t} P_t(\sigma_t(i))$; tail emits no bits. 4. Inside top-n_t, normalize P_t and recursively split ranks into even/odd groups. 5. Run BinEnc on group masses; consume a coded bit only when the selected path enters the odd group. 6. Advance unused DRBG calls to the fixed budget $\lceil \log_2 n_t \rceil$. **Output:** selected behavior $\hat{b}_t$ and consumed coded bits.	**Input:** $\hat{b}_t$, verifier top-k order σ'_t, context h_t, key $K_{\rm sh}$. 1. Set $n_t = \min(k, \vert\sigma'_t\vert)$ and derive the same DRBG sequence. 2. If $\hat{b}_t \notin \sigma'_t[1 : n_t]$, return no bits. 3. Let q be the zero-indexed rank of $\hat{b}_t$ in σ'_t. 4. Reconstruct the path by parity: odd q means the encoder entered the odd group and emitted one bit. 5. Decode that bit from the corresponding DRBG draw; set $q \leftarrow \lfloor q/2 \rfloor$ and repeat. 6. Consume exactly $\lceil \log_2 n_t \rceil$ DRBG calls, matching the encoder. **Output:** decoded coded bits for step t.

Shared alignment. Both sides share $(K_{\rm sh}, h_t)$ and the fixed DRBG budget; only the encoder needs calibrated probabilities. Correct decoding requires the verifier to reconstruct the same top-k rank path, after which RLNC pools decoded bits across the audit window.

Figure 2. One-step rank codec. The encoder uses probability masses only to sample from the original behavior policy, while the verifier decodes from the selected behavior's top-k rank path.

trajectory is long enough or the audit window spans enough steps.

4.7. Verification Protocol

An audit takes as input a pre-registered key/payload claim, a canonicalized trajectory log, and a verifier-side procedure for reconstructing top-k candidate orders. The verifier first derives the per-step DRBG streams from logged contexts and decodes all observed steps using RankDec. It then runs RLNC elimination over the recovered coded equations. If the system recovers the registered payload and the reconstructed ranks pass a deployment-specific quality gate, the audit reports a positive attribution. If the payload does not recover but the rank-quality gate fails or too few independent equations are available, the audit reports *inconclusive*. A negative attribution is reserved for the case where the verifier has enough rank-consistent evidence to decode but the recovered payload contradicts the registered claim.

The rank-quality gate is intentionally separate from the watermark statistic. A verifier can estimate it using held-out calibration contexts, duplicate model queries, or agreement between independent rankers. This prevents a common operational error: treating a failed rank-only decode as evidence that a trajectory is clean when the verifier's channel reconstruction is itself unreliable.

Before an audit window has accumulated enough independent RLNC equations, the verifier can still estimate whether the recovered equations are informative under a binomial null model of random agreement. This diagnostic can trigger further collection or manual review, but it is not by itself a positive attribution decision.

Appendix A expands the distribution-preservation proof, Appendix B details the fixed-budget synchronization rule, and Appendix C summarizes how decoded bits become payload equations. These appendices are not separate claims; they make the construction auditable without interrupting the main channel-knowledge argument.

The top-k parameter controls a reliability–capacity tradeoff. Larger k exposes more rank information, but it also asks the verifier to reconstruct lower-probability alternatives whose order may be less stable across models and serving conditions. Smaller k is more conservative and better aligned with restricted APIs, but it leaves less room for embedding. We therefore evaluate a sweep rather than tuning to a single universal value; a deployment should choose k from calibration data by maximizing expected payload recovery subject to a rank-agreement threshold.

5. Security Analysis

Proposition 1 (distribution preservation). For each step t and behavior $b \in \mathcal{B}_t$, AsymMark-R outputs b with probability $P_t(b)$.

Proof sketch. The top/tail gate selects the visible top-n_t region with probability equal to its total mass and otherwise samples the tail proportionally. Tail samples emit no coded bits, so the verifier need not reproduce the gate randomness. Conditioned on entering the visible region, each binary split uses BinEnc, whose selected group probability equals its normalized mass because r_m is uniform. Induction over the partition tree gives each leaf its normalized probability within top-n_t. Multiplying by the visible-region mass recovers $P_t(b)$. □

Lemma 1 (probability-bin instability). There exist two behavior distributions with identical rank order for which a probability-dependent bin decoder assigns the same selected behavior to different bins.

Proof sketch. Consider four behaviors ordered $a \succ b \succ c \succ d$ and a two-bin prefix decoder whose first bin is the smallest rank prefix whose cumulative probability exceeds $1/2$. Under $P = (0.45, 0.30, 0.15, 0.10)$, the first bin is $\{a, b\}$. Under $Q = (0.55, 0.25, 0.12, 0.08)$, the rank order is unchanged but the first bin is $\{a\}$. The selected behavior b is therefore interpreted in different bins although the verifier preserved the full ranking. Differential recombination uses a richer bin structure than this toy prefix rule, but it shares the same dependence on probability masses; rank agreement alone does not fix its decoder partition. □

Proposition 2 (rank-noise robustness boundary). Under an adjacent-swap noise model with per-adjacent-pair swap probability ϵ, suppose a selected rank's decoded path of depth $d_t = \lceil \log_2 n_t \rceil$ can be corrupted by at most cd_t

relevant local swaps, where $n_t = \min(k, |\mathcal{B}_t|)$ is the actual visible candidate count. Then the per-step path-corruption probability is bounded by

$$p_{\text{flip},t} \leq 1 - (1-\epsilon)^{c\lceil \log_2 n_t \rceil}. \tag{8}$$

Combining this bound with the coded-evidence model below gives the observation budget needed to distinguish the watermark from the null at a chosen tail probability. The bound is deliberately model-dependent: arbitrary targeted rank manipulation remains outside the weak-asymmetric guarantee, but the expression captures the empirical tradeoff that larger k gives more path bits while exposing more rank positions to noise. When all steps fill the requested top-k view, $\lceil \log_2 k \rceil$ is the corresponding upper-bound depth.

Corollary 1 (zero-noise rank preservation). Let P_t and P'_t satisfy $\mathrm{argsort}_k(P_t) = \mathrm{argsort}_k(P'_t)$. For any selected behavior $\hat{b}_t$ and shared key/context-derived DRBG stream, RankDec returns the same bits under both distributions. This follows immediately from Proposition 2 with $\epsilon = 0$: the decoder accesses channel information only through the rank position of $\hat{b}_t$, and the DRBG stream depends on the key and context rather than probability values.

Corollary 2 (coded-bit evidence under rank agreement). Let ρ be the probability that an observed embedding step is top-k rank-consistent between embedder and verifier. Suppose rank-consistent steps decode a coded bit correctly with probability $q > 1/2$, while rank-inconsistent steps are independent of the claimed bit. Then the expected coded-bit agreement rate is

$$\mathbb{E}[\bar{X}] = \frac{1}{2} + \rho(q - \frac{1}{2}). \tag{9}$$

For N decoded bits, Hoeffding's inequality gives

$$\Pr\left[\bar{X} \leq \frac{1}{2} + \gamma\right] \leq \exp\left(-2N\left(\rho(q-\frac{1}{2}) - \gamma\right)^2\right) \tag{10}$$

whenever $0 < \gamma < \rho(q - \frac{1}{2})$. Thus lower rank agreement does not create a hard impossibility; it increases the evidence budget needed to distinguish the watermark from the null. This corollary is a diagnostic model for coded-bit evidence, not a replacement for pooled payload recovery.

Proposition 3 (packet scarcity). For an L-bit deterministic RLNC payload, recovery over an audit window requires at least L independent coded packets. If M independent packets survive after top-k misses, erasures, and rank failures, then

$$\Pr[\text{recover}] \leq \Pr[M \geq L]. \tag{11}$$

For an audit window pooling trajectories $i = 1, \ldots, n$, the expected packet budget is additive:

$$\mathbb{E}[M_{\text{pool}}] = \sum_i \mathbb{E}[M_i]. \tag{12}$$

If each trajectory has expected packet yield $\mu_{\text{traj}} = N(1 - r)\mathrm{DSR}(k, \epsilon)$ under erasure rate r and rank-noise level ϵ, then a deployment aiming for L independent equations plus a rank-deficiency margin δ needs, in expectation,

$$n \gtrsim \frac{L + \delta}{\mu_{\text{traj}}}. \tag{13}$$

This explains why short ToolBench traces can provide useful packet evidence while requiring cumulative audit windows for payload recovery: pooling trajectories converts sparse per-trace packets into enough independent equations.

Proposition 4 (top-k marginal gain and saturation). Let $C_{\text{nom}}(k)$ be the nominal coded-bit opportunity exposed by a top-k verifier view, and let $\mathrm{DSR}(k)$ be the observed decode success rate under that view. Increasing k can raise $C_{\text{eff}}(k) = C_{\text{nom}}(k)\mathrm{DSR}(k)$ in two ways: by exposing more coded bits, and in packet-limited regimes by pushing more trajectories over the minimum packet threshold for payload recovery. Once additional ranks mostly add unstable low-probability candidates, marginal gains should saturate and may turn negative. This statement does not imply an internal optimum in every measured range. If the measured k values do not reach the regime where rank instability dominates, $C_{\text{eff}}(k)$ can be monotone over the tested interval and the best observed k is only a lower bound on the true operating point.

Minimum evidence threshold. For bit-level triage before pooled RLNC recovery, a one-sided binomial test with match probability $q > 1/2$ under the watermark and $1/2$ under the null needs on the order of

$$N_{\min} = O\left(\frac{z_\alpha^2}{(q - 1/2)^2}\right) \tag{14}$$

observed decoded bits to reach significance level α. This threshold is not an attribution decision by itself; it sizes the observation window needed before a verifier should expect enough channel evidence to justify attempting pooled payload recovery.

False positives. For an unwatermarked or unrelated audit window, positive attribution requires recovery of a key and payload claim that was registered before inspection. Under the standard null that decoded clean bits are independent of this fixed claim and that no adaptive post-hoc key or payload search is counted as a single test, an L-bit payload match occurs with probability 2^{-L}. In practice the verifier can tune L to the desired audit threshold.

Why the theory matches the contribution. Lemma 1 gives the negative result: rank agreement alone is insufficient for probability-bin decoders. Proposition 2 is the quantitative robustness statement for AsymMark-R under rank noise, with Corollary 1 giving the zero-noise rank-preservation case. Proposition 3 separates channel robustness from evidence volume, which is why the evaluation reports payload recovery as pooled audit-window recovery rather than isolated trajectory success. Proposition 4 explains why top-k should be calibrated rather than treated as a universal constant: larger k can raise nominal capacity, but the measured optimum is only meaningful once the sweep reaches the regime where rank instability erodes decode success.

Key privacy. The deployment watermark key is used only to derive context-specific pseudorandomness. The public trajectory and rank list do not reveal the key under the pseudorandom function assumption for HMAC-SHA512. A verifier may check many trajectories under the same registered owner only with domain-separated contexts, key identifiers, and tenant/product isolation; operational deployments should rotate keys to limit blast radius if a key is disclosed. The offline experiments use recorded payload metadata and deterministic replay to evaluate the watermark channel; they should not be read as evidence that a public context-derived prototype default provides secret-key security.

Claim binding. A watermark match is meaningful only for a claimed key and payload that were bound to an owner before the audit. Otherwise, an operator could test many keys or payloads after observing a trajectory and inflate the effective false-positive rate. Deployments should therefore register a key identifier, payload namespace, and issuance time in an append-only registry or signed manifest. The verifier can then count only pre-registered claims and apply the multiple-testing correction in Section 6.13.

Limits. AsymMark-R is robust to value perturbations that preserve top-k order, not to arbitrary rank manipulation. If an adversary can force targeted rank swaps in the top-k list, it can corrupt decoded paths. This is the expected boundary of any rank-only weakly asymmetric scheme. The evaluation therefore includes a rank-disagreement proxy and treats live cross-model rank reconstruction as deployment work. Appendix D states the corresponding rank-stability intuition and the resulting top-k choice rule.

6. Evaluation

6.1. Setup

We evaluate on ALFWorld [8] and ToolBench [9]. ALFWorld tests embodied planning over in-distribution (ID) and out-of-distribution (OOD) splits; ToolBench tests tool-use behavior across six split categories. The LLM endpoints are DeepSeek Chat (`deepseek-chat`) [14] and Gemini 2.0 Flash (`gemini-2.0-flash`) [3]. The postprocessed corpus contains 11,820 trajectories and 4,728 watermark decode-capable trajectories. Methods are: vanilla LLM agent, clean AgentMark pipeline without embedding, red-green (RG) behavior bias, symmetric AgentMark-F, and rank-based AsymMark-R. RG follows the Agent Guide style of keyed behavior steering [15]; it is included as a high-level contrast to distribution-preserving watermarking rather than as a tuned competitor.

Unless otherwise stated, recovery rates are computed from the available offline decoder artifacts. Capacity tables are bit-level channel proxies over logged trajectories and should not be conflated with deployment attribution. Payload recovery is reported as cumulative audit-window recovery: packets from trajectories in the same dataset/model/split/run/configuration are combined before RLNC decoding. This matches the pooled recovery curve in Fig. 3 and treats logged trajectories as packet sources for the same cumulative audit window. Deployment attribution should use registered payloads sized to the required false-positive threshold and the false-positive accounting in Section 6.13.

Evaluation coverage after postprocessing is balanced across the two watermarking methods. ALFWorld contributes four model/split groups with 3/3 completed runs per group and 3,288 decode-capable trajectories, split evenly as 1,644 per watermark. ToolBench contributes two model/split groups with 18/18 completed runs per group and 1,440 decode-capable trajectories, split evenly as 720 per watermark. Utility and stealth metrics use the available methods, with vanilla reported separately in Appendix F; decode-capable counts apply only to the two watermarking methods because vanilla, clean, and RG trajectories do not carry payload metadata.

6.2. Protocol

The evaluation pipeline starts from logged trajectories containing the selected behavior, candidate list, behavior probabilities when available, and watermark metadata. Utility and JSD are computed over all valid task trajectories. Watermark recovery is computed on the decode-capable subset: trajectories with a recorded watermark configuration, a recoverable candidate ordering, and enough logged context to replay the deterministic DRBG schedule. This distinction is reported explicitly in the corpus counts above.

For each decode-capable trajectory, we run the same verifier under several channel views. The *exact* view uses the logged probabilities as a diagnostic upper bound rather than as the target audit interface. The *top-k* views discard numeric probabilities and retain only the ordered candidate list. The selected-only view is used as a negative control when available. Rank-noise experiments use a bit-level disagreement proxy parameterized by ϵ because the postprocessed logs do not contain full alternate rank lists for every verifier; erasure experiments remove a random fraction of observed steps before RLNC solving. All tables aggregate over the available model/split groups rather than selecting the best configuration for each task. The main rank-only comparison reports fixed top-10, top-6, and top-2 views; the separate top-k ablation reports the full sweep and is not used to choose per-model winners. These aggregates are descriptive comparisons over the generated corpus; they are not presented as hypothesis tests for small differences between methods.

6.3. Reproducibility

The evaluation package is organized so that each reported number is tied to a reproducible postprocessing artifact. The main result package supports utility, stealth, verifier-channel capacity, pooled recovery, and evidence-strength claims; the robustness package supports rank-noise, erasure, matched top-k recovery, live cross-model reranking, and false-positive checks. Appendix G summarizes this mapping at the claim level. We treat these artifacts as a reproducibility trail for the reported diagnostics; the live ToolBench L5 rerank results are deployment-facing evidence for the verifier channel rather than a claim that all agent domains have equally stable cross-model ranks.

6.4. Evaluation Questions

The evaluation is organized around four questions rather than a table-by-table result dump. First, how much usable capacity survives when verifier channel knowledge degrades from exact probabilities to ranks? Second, does rank decoding preserve task utility and behavior-frequency statistics? Third, how does the scheme behave under rank noise, erasure, and cross-model reranking? Fourth, what evidence threshold is needed before a recovered payload should be treated as attribution rather than channel diagnostics?

6.5. Metrics

We report task success rate and average steps for utility. We report Jensen-Shannon divergence (JSD) between empirical action-frequency distributions for a coarse stealth check. For watermark recovery, *payload recovery* is the fraction of pooled audit windows whose decoded payload exactly matches the embedded payload under the payload convention defined in the setup. When a verifier view does not recover the registered payload in a window, its payload recovery for that window is zero. We also report proxy C_{eff} in units of effective proxy bits per task trajectory:

$$C_{\text{eff}}^{\text{proxy}} = C_{\text{nom}}^{\text{proxy}} \cdot \text{DSR}^{\text{proxy}}. \tag{15}$$

Here $C_{\text{nom}}^{\text{proxy}}$ is the mean decoded proxy-bit contribution from each logged trace, and $\text{DSR}^{\text{proxy}}$ is proxy payload recovery over the pooled audit window. Thus $C_{\text{eff}}^{\text{proxy}}$ is a per-trajectory effective-capacity diagnostic; a per-step version would divide it by the mean trajectory length. This score is useful for comparing channel views because a high-capacity exact-channel scheme that fails under rank-only verification is scored as low usable capacity in that verifier channel. For confidence analysis, we use a one-sided binomial tail probability under the null hypothesis that each decoded coded bit agrees with the claim with probability $1/2$.

6.6. Verifier-Channel Diagnosis

The central empirical comparison is not simply whether AsymMark-R has higher recovery than AgentMark-F in every channel. It does not: exact probabilities are the native channel for AgentMark-F, and Table 1 shows that AgentMark-F has higher exact-channel C_{eff} on ALFWorld. The question is whether that exact-channel advantage survives the weaker verifier interface that motivated this paper. The $L1$ capacity column and the channel-ladder figure test precisely this mechanism. If a decoder depends on probability bins, Lemma 1 predicts that preserving candidate order is not enough to preserve decoder state. If a decoder depends only on rank path, Corollary 1 predicts no loss from probability-value perturbations as long as ranks are preserved. The evaluation therefore treats the exact-probability view as an upper-bound capacity diagnostic and the top-k rank view as the target robustness channel.

6.7. Utility and Behavioral Stealth

Table 1 summarizes the main result along the three quantities needed for the channel-knowledge story: task utility, behavior-frequency stealth, and effective capacity under exact versus rank-only verifier views. On ALFWorld, AsymMark-R reaches 78.6% success versus 80.0% for clean and 79.4% for AgentMark-F. On ToolBench, AsymMark-R reaches 77.1% versus 75.7% for the clean pipeline. We interpret these aggregates conservatively: rank encoding stays close to the clean and symmetric pipelines and does not exhibit the large utility tax associated with naive behavior bias, but small one- or two-point differences are not treated as significant. The capacity columns are shown only for payload-carrying methods and report $C_{\text{eff}}^{\text{proxy}}$ per task trajectory. The retained column is the ratio $L1/L0$, making the verifier-channel degradation explicit; it should be read together with the absolute $L1$ capacity because high retention on a low-capacity domain does not imply higher usable capacity.

6.8. Rank-Only Verification

Under exact probabilities on ALFWorld, AgentMark-F has higher payload recovery (0.700) than AsymMark-R (0.532), reflecting the efficiency cost of rank-only structure. Under top-10 rank-only verification, AgentMark-F collapses to 0.014 payload recovery, while AsymMark-R remains at 0.068. In C_{eff} terms, ALFWorld AgentMark-F falls from 19.692 under exact probabilities to 0.333 under top-10 ranks, while AsymMark-R falls from 7.707 to 1.000. Figure 4 plots the same ladder as relative retained capacity, normalizing each method/domain series by its own L0 value so that both benchmark panels show the degradation shape while the absolute C_{eff} values remain in Table 1.

We also add the step-level counterfactual suggested by Lemma 1. For each AgentMark-F step whose exact decoder emits bits, we keep the top-k order fixed, drop candidates outside the verifier's top-k view, renormalize the remaining probabilities, and re-run the same context-derived decoder on the same selected behavior. This is a truncation-and-renormalization instance of the value perturbations in Lemma 1, not an exhaustive test of all possible calibrated-probability changes. When the selected behavior remains inside top-k, any changed bin or changed bits are therefore probability-bin effects rather than rank-order effects; the bin-change and bit-change percentages below are conditional on that selected-in-top-k event, while selected-dropped rates are measured over exact-decodable steps. On ALFWorld DeepSeek, among rank-stable steps, top-10 still changes the probability bin in 13.7% of steps and flips decoded bits in 11.1%; top-4 changes bins in 56.8% and flips bits

TABLE 1. Main utility, stealth, and verifier-channel capacity summary. Success, steps, JSD, and C_{eff}/task report mean±standard deviation over the available aggregation cells; lower JSD is closer to clean behavior frequencies. $L0$ is exact-probability verification, $L1$ is top-10 rank-only verification, C_{eff}/task is the proxy effective capacity per task trajectory, and Retained is the ratio of means $L1/L0$. RG denotes red-green behavior bias.

Dataset	Method	SR (%) ↑	Steps ↓	JSD ↓	C_{eff}/task $L0$ ↑	C_{eff}/task $L1$ ↑	Retained $L1/L0$ ↑
ALFWorld	Clean	80.0±10.5	16.23±2.19	0.000	–	–	–
	RG	74.9±13.1	17.58±2.75	0.134±0.061	–	–	–
	AgentMark-F	79.4±10.7	16.58±1.89	0.122±0.079	19.692±8.109	0.333±0.168	1.7%
	AsymMark-R	78.6±10.0	16.40±2.11	0.121±0.070	7.707±4.590	1.000±0.730	13.0%
ToolBench	Clean	75.7±2.4	5.32±0.78	0.000	–	–	–
	RG	75.1±3.2	4.78±0.01	0.181±0.069	–	–	–
	AgentMark-F	74.7±0.2	4.54±0.04	0.174±0.077	0.248±0.182	0.009±0.020	3.6%
	AsymMark-R	77.1±0.5	5.07±0.77	0.140±0.058	0.053±0.038	0.050±0.036	94.3%

in 51.3%. On ToolBench DeepSeek, top-10 is milder but nonzero (4.6% bin changes, 3.7% bit changes), while top-4 again exposes a large bin-instability regime (52.6% bin changes, 43.6% bit changes). This turns the earlier aggregate observation into a direct per-step attribution: exact-channel probability bins are unstable even when the selected behavior and retained top-k rank order are unchanged.

ToolBench is lower-capacity for both schemes because trajectories are short and many tool choices have low recoverable entropy. Nevertheless, AsymMark-R keeps nearly the same payload recovery from exact probabilities to top-10 rank-only decoding (0.033 to 0.032), whereas AgentMark-F drops from 0.085 to 0.003 payload recovery. The same pattern appears in C_{eff}: ToolBench AgentMark-F drops from 0.248 to 0.009, while AsymMark-R remains effectively flat from 0.053 to 0.050.

AsymMark-R weakens exact probability knowledge to rank knowledge; it does not claim attribution from the action log alone. A selected-behavior-only verifier recovers no payload in these diagnostics, so we keep it as a negative-control sanity check rather than a main result column.

The same mechanism appears more clearly in pooled audit windows. Figure 3 compares ALFWorld/DeepSeek/ID top-10 recovery as the verifier increases the pooled trajectory budget within the same audit configuration. AgentMark-F reaches perfect recovery by pool size 5 when exact probabilities are available, confirming that the baseline is not packet-limited in its native channel. Under a rank-only surrogate that keeps only the top-10 order and assigns monotone linear pseudo-probabilities before running the AgentMark-F differential decoder, pooling does not rescue the baseline: recovery remains between 0.013 and 0.247 over the plotted window. In contrast, AsymMark-R rises to 0.613 at pool size 10, 0.780 at 20, and 0.807 at 50. The point is not that AsymMark-R dominates exact-channel AgentMark-F; it is that the rank-only decoder matches the verifier channel, while a probability-bin decoder continues to accumulate conflicting equations even when the audit window grows.

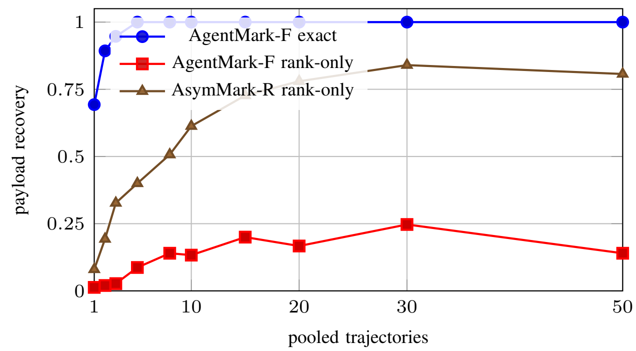

Figure 3. Rank-only pooled recovery on ALFWorld/DeepSeek/ID, top-10 verifier view. The plotted window stops at 50 trajectories to show the audit-window regime before very large pools require conflict-aware aggregation.

6.9. Top-k Calibration

Increasing k raises both the amount of available rank information and the risk of rank instability, matching Proposition 4's saturation view. Figure 5 reports the matched-top-k ToolBench reruns where the embedder and verifier use the same rank depth and RLNC recovery is evaluated on actual generated trajectories. Single trajectories almost never recover the 8-bit payload because ToolBench traces are short and packet yield is sparse. Pooling trajectories into larger audit windows shows the operating regime directly: very small k can be packet-limited, while moderate and larger k provide enough independent packets for same-model recovery.

This calibration is a real decoding result, not the offline capacity proxy used for Table 1. We keep the ALFWorld top-k sweep out of this figure because the local ALFWorld artifact is an offline replay over logged rank traces rather than a matched-top-k rerun. The corresponding proxy still helps explain candidate-set saturation, but it should not be read as task-level or audit-window payload recovery.

6.10. Rank Noise and Erasure

The robustness experiments use pooled recovery throughout. Pooled recovery combines packets from trajec-

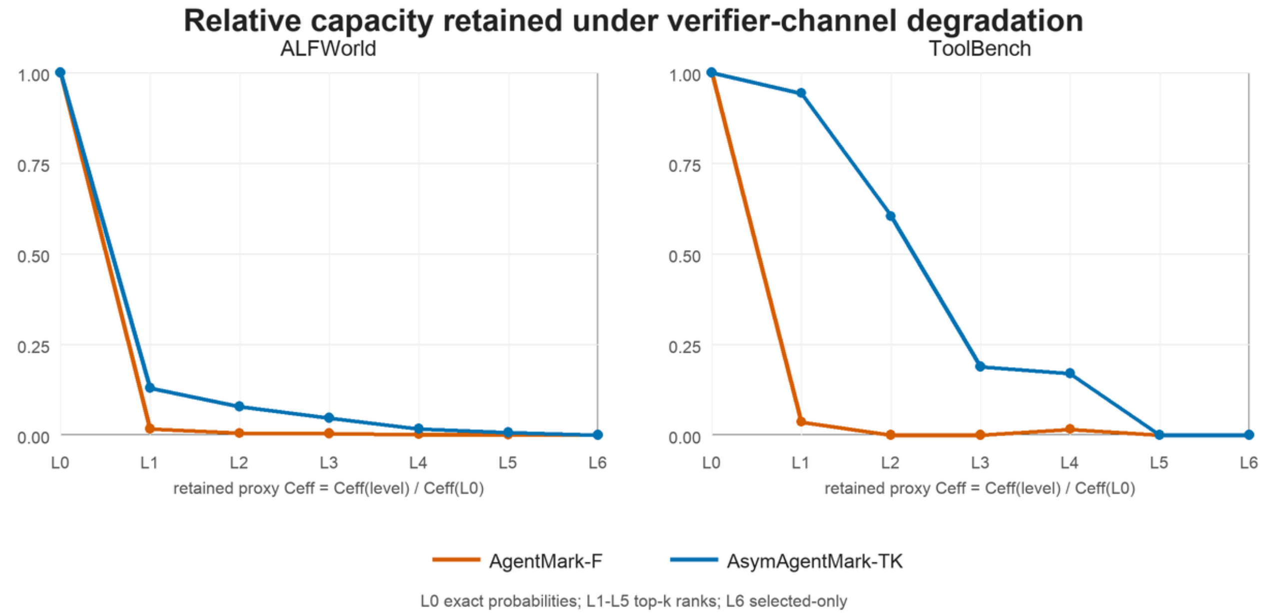

Figure 4. Relative effective capacity retained over the L0–L6 verifier-channel ladder. The two panels separate ALFWorld and ToolBench because their absolute proxy C_{eff} scales differ by almost two orders of magnitude; each curve is normalized by its own L0 exact-probability value. AgentMark-F loses usable capacity as soon as the verifier only has ranks, while AsymMark-R degrades more gradually under top-k rank views.

tories with the same dataset, model, split, run, and verifier configuration before RLNC decoding. This corresponds to corpus-level auditing or a longer audit window, not to a task-level success rate for isolated trajectories.

The pooled rank-noise and erasure sweeps are intentionally conservative audit-window checks, and they mostly saturate: under DeepSeek top-10 adjacent-swap noise, ALFWorld ID and ToolBench all-split pooling remain at 1.0 through $\epsilon = 0.5$, while ALFWorld OOD is the only nontrivial cell, dropping to 0.83 at $\epsilon = 0.5$. Under fixed erasure, both watermarking methods recover in every pooled cell even at 90% step erasure. Appendix E records this saturation and reports a medium-window path-consistency diagnostic that avoids the all-pooled endpoint; the main text focuses on the non-pooled path-flip curve below.

To test Proposition 2 on the quantity it bounds, Fig. 6 directly measures the empirical decoded-path flip rate for decodable ToolBench steps under adjacent swaps, and overlays an empirical envelope of the form $1-(1-\epsilon)^{c\lceil\log_2 n_t\rceil}$. The fitted envelope validates the shape and depth dependence of the boundary, not a prior prediction of a universal constant. The main takeaway is qualitative: deeper rank paths expose more branch decisions to local rank disagreement, so path-corruption risk rises with both rank noise and candidate depth.

6.11. Semantic Rewrite Robustness

Rank perturbations isolate the channel mechanism, but a verifier may also face behavior serializations that are semantically close while textually changed. Table 2 reports the non-window diagnostics for the ToolBench semantic-rewrite rerun under the top-10 verifier view, while Fig. 7 plots the audit-window recovery curve. The rewrite strength changes the surface form substantially: token Jaccard falls from .532 under light rewrites to .271 under heavy rewrites. The exact rank path is therefore not preserved at the step level, and single trajectories almost never recover the 8-bit payload. However, pooled recovery rises as the verifier accumulates surviving packets, reaching 1.0 for all three strengths when the full 183-trajectory audit window is used.

6.12. Top-k Audit Calibration

We also run ToolBench cross-model calibration experiments in which the verifier reconstructs top-k ranks with a different model from the embedder. The same-model curve in Fig. 5 checks whether the rank codec has enough packet yield under matched conditions; the cross-model rerank checks whether the verifier model preserves the rank paths well enough for pooled recovery.

Figure 8 summarizes the cross-model audit-window calibration. Cross-model verification is more selective than the same-model reruns: small k can be packet-limited, while larger k can accumulate conflicting packets as lower-ranked candidates become less stable across models. The successful depths are direction-dependent, with DeepSeek-to-Gemini favoring a shallower rank view and Gemini-to-DeepSeek favoring a slightly deeper one.

Figure 8 makes the mechanism visible. Increasing k can increase packet yield, but it also asks the verifier to reconstruct lower-ranked candidates whose rank paths are

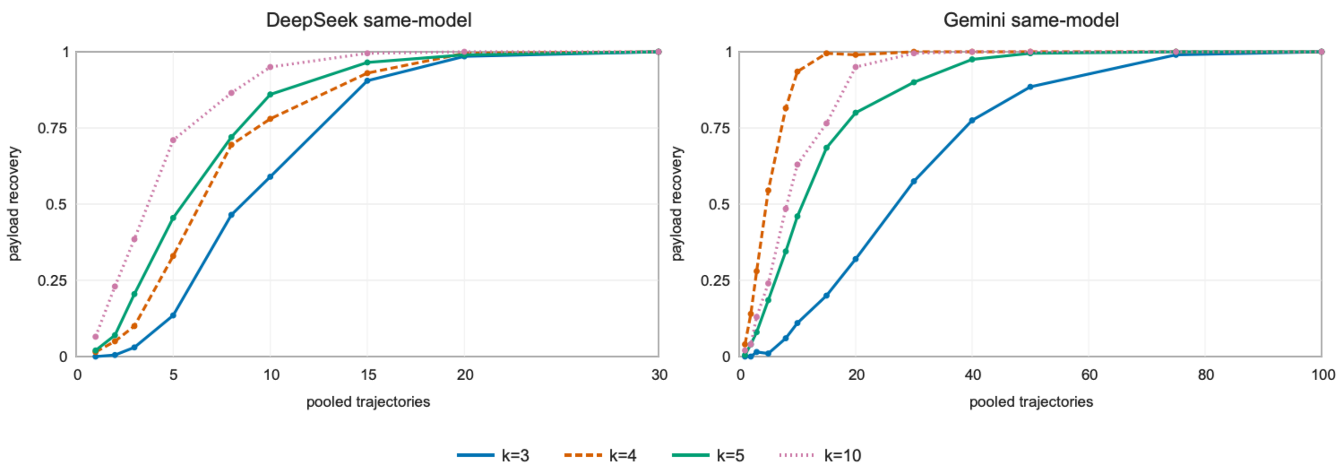

Figure 5. ToolBench matched-top-k same-model real decoding calibration for AsymMark-R. Curves show sampled pooled payload recovery for $k \in \{3, 4, 5, 10\}$ as the audit window grows; each panel uses trajectories generated and verified by the same model, with panel-specific x-axis ranges to show the different saturation regimes.

TABLE 2. SEMANTIC REWRITE DIAGNOSTICS FOR ASYMMARK-R ON TOOLBENCH, DEEPSEEK, TOP-10. SIMILARITY AND RANK METRICS ARE STEP-WEIGHTED OVER 359 REWRITTEN STEPS PER STRENGTH; STRICT DSR IS SINGLE-TRAJECTORY PAYLOAD RECOVERY OVER 183 TRAJECTORIES; PACKETS/CONFLICTS ARE MEASURED ON THE FULL POOLED AUDIT WINDOW.

Rewrite	ROUGE-L	Jaccard	Rank keep	Top-1 match	Top-10 overlap	Strict DSR	Packets/conflicts
Light	.519	.532	.306	.393	.593	.011	45/14
Medium	.304	.316	.309	.379	.593	.011	45/13
Heavy	.272	.271	.312	.409	.594	.011	45/14

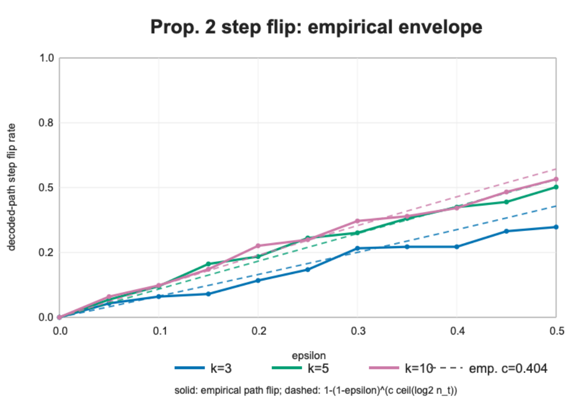

Figure 6. Fine-grid ToolBench rank-noise diagnostic for Proposition 2. Solid lines show empirical decoded-path step-flip rates under adjacent swaps; dashed lines show an empirical envelope $1 - (1 - \epsilon)^{c\lceil \log_2 n_t \rceil}$ with fitted $c = 0.404$.

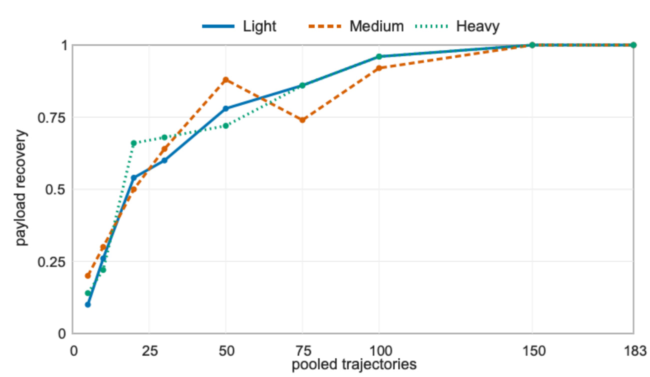

Figure 7. Semantic-rewrite pooled recovery for AsymMark-R on ToolBench, DeepSeek, top-10. Curves show sampled pooled payload recovery as the audit window grows; the final point pools all 183 trajectories and matches the packets/conflicts endpoint in Table 2.

less stable across models. In the rank tree, even a one-position displacement can move a selected behavior to a different parity branch and turn a useful RLNC equation into a conflicting one. Thus the relevant quantity is not nominal depth alone; it is the balance between independent packet yield and rank-path consistency.

The same live L5 protocol exposes a boundary on ALFWorld. Cross-model top-10 overlap is moderate (0.596 for DeepSeek-to-Gemini and 0.633 for Gemini-to-DeepSeek), but top-1 agreement is only about 0.20, and pooled recovery succeeds only for the Gemini-to-DeepSeek ID pool in the pulled artifact. We therefore treat cross-model rerank as a ToolBench-positive but domain-dependent result, not as evidence that every agent domain has stable enough ranks

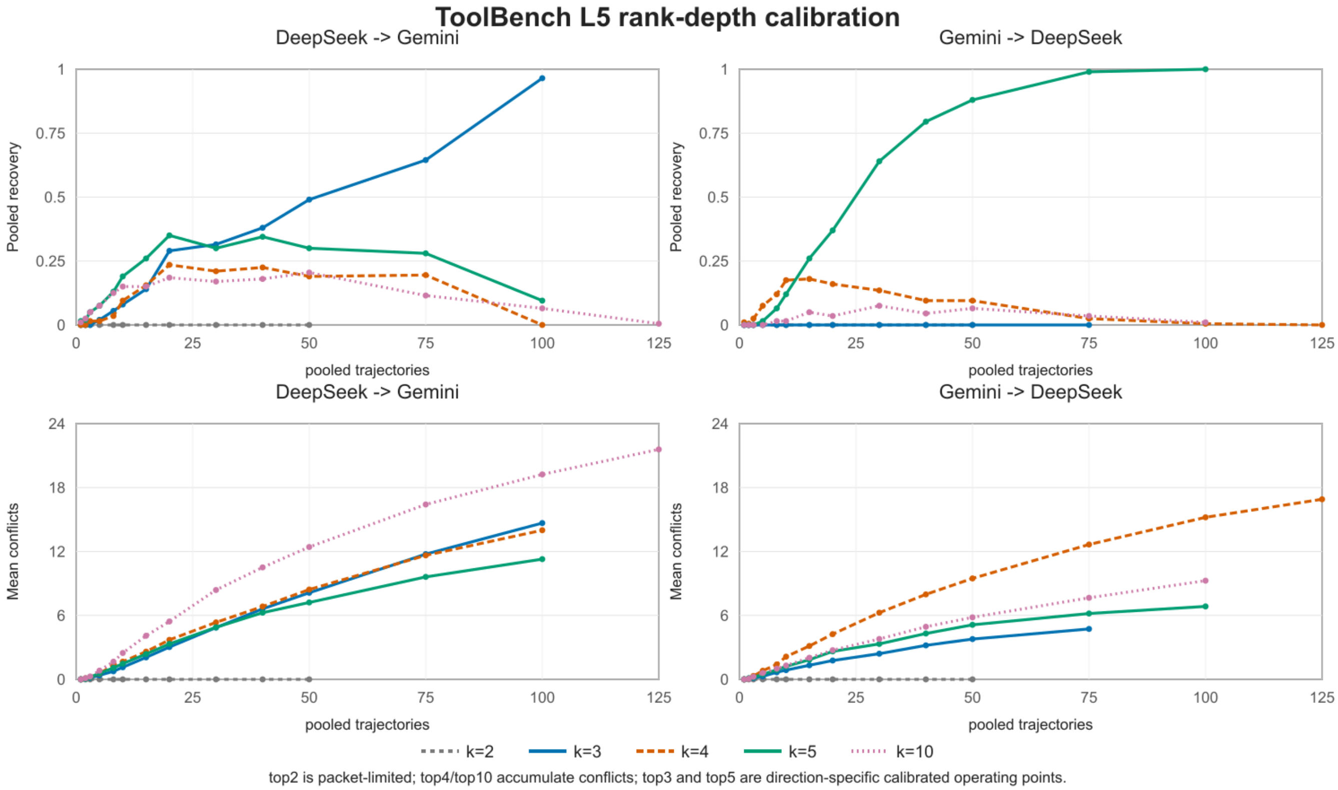

Figure 8. Live ToolBench L5 cross-model rank-depth calibration. The top row shows sampled pooled recovery; the bottom row shows mean packet conflicts over the same pools. Small k is packet-limited, while larger k can accumulate conflicts as lower-ranked candidates become less stable across verifier models.

for pooled recovery without calibration.

6.13. False Positives and Evidence Thresholds

False-positive control is analytic for exact payload claims. If a clean or unrelated audit window yields coded bits that are independent of a fixed pre-registered payload claim, and no adaptive post-hoc key or payload search is counted as the same test, the chance that it recovers that exact L-bit payload is 2^{-L}. Operational deployments should choose registered payload lengths according to the required audit threshold and available trajectory length.

The threshold must also account for multiple testing. If an operator tests N independent key/payload claims against the same corpus, a conservative union bound gives a family-wise false-positive rate at most $N2^{-L}$. This is why payload length is an audit-policy parameter rather than an implementation detail: larger claim spaces are needed when many owners, keys, or payloads may be tested against the same corpus.

Empirical false-positive trials follow this analytic curve across the tested datasets, models, and top-k configurations. Because the attribution event is exact payload equality for a pre-registered claim, we report the analytic per-claim threshold 2^{-L} as the security control and use random-claim trials only to check that the implementation behaves like the null model. Appendix G maps this and the other reported diagnostics at the claim level.

Bit-level scores require a different interpretation. When a verifier reports a match rate over n decoded coded bits rather than a recovered payload, the appropriate null is a binomial tail $\Pr[X \geq s; X \sim \mathrm{Binom}(n, 1/2)]$ for the observed score s. This statistic is useful for channel diagnostics and triage before enough independent RLNC equations are available, but it is not a positive attribution decision unless the pre-registered payload itself is recovered.

The confidence-curve diagnostics in Fig. 9 illustrate this distinction: bit-level evidence can become statistically strong before a full payload is recovered, but it still quantifies channel evidence rather than ownership by itself. It becomes attribution evidence only when the same audit also satisfies claim binding, rank-quality gating, and pooled payload recovery.

7. Discussion

Exact probability is a luxury. The evaluation supports a pragmatic conclusion: if exact P_t is available, symmetric AgentMark-F can be more capacity-efficient. If the verifier has only top-k ranks, symmetric decoding becomes unreliable, and a rank-decodable construction is the relevant design point.

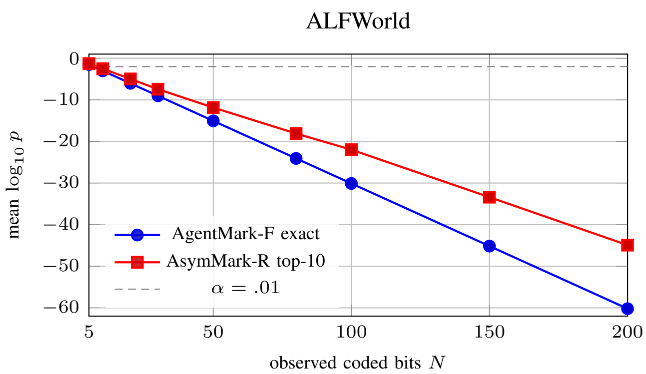

Figure 9. Bit-level evidence confidence under the binomial null. Curves aggregate completed model/split cells and report the mean one-sided $\log_{10} p$ value as the verifier observes more decoded coded bits. These curves size evidence windows; positive attribution still requires pre-registered payload recovery.

ToolBench exposes a low-entropy regime. Tool-use traces are shorter and often contain fewer semantically viable actions than embodied planning. AsymMark-R preserves utility in this setting, but payload recovery depends on accumulating enough packets across an audit window. This suggests combining rank encoding with adaptive step selection, longer audit windows, or higher-level tool-plan aggregation.

Rank consistency is observable. Unlike exact probability equality, rank consistency can be measured directly during verification. A verifier can report top-k agreement, Kendall tau distance, or adjacent-swap rates to calibrate confidence.

Rank is one invariant, not the whole framework. AsymMark is defined by the separation between a policy-preserving embedding channel and a verifier-reconstructible decoding invariant. Top-k rank order is the invariant we implement and evaluate because it is naturally exposed by restricted agent APIs and sufficient to recover the interleaved partition path. Section 4.5 gives a second constructive instance: an unordered-set keyed partition that decodes from set membership plus keyed canonicalization on a balanced synthetic channel. This does not replace the rank system in our experiments, but it demonstrates that the framework is not merely a wrapper around a single rank primitive. Other candidate invariants could instantiate the same template in future work, including coarse quantized probability buckets, signs of pairwise comparisons, and Kendall-style order constraints. These alternatives may trade capacity, stability, and verifier cost differently; this paper focuses on the rank instance because it gives a clean first operating point on real agent benchmarks.

Why not fully asymmetric verification? A verifier that sees only the selected action has too little information to distinguish a keyed watermark from ordinary policy variation without shifting the agent's behavior distribution. The selected-only negative-control view in our evaluation reflects this limit. AsymMark-R therefore takes a middle position: it does not require calibrated probabilities, but it does require a reproducible candidate ranking. This is a practical assumption for many audited agents because tool schemas, admissible environment actions, and candidate lists are often easier to reconstruct than exact sampling probabilities.

Adaptive adversaries. An adversary aware of the watermark could try to perturb ranks while preserving task success. This attack is outside pure distribution perturbation and should be treated as an active removal attack. Defenses include randomized k, context-dependent rank partitions, and cross-step consistency checks.

Operational audit flow. A practical verifier should not treat every low-confidence trajectory as a negative attribution. Instead, it should first check whether the top-k rank reconstruction is itself reliable. If the reconstructed rank agreement is poor, the correct conclusion is an inconclusive audit rather than a clean verdict. This distinction matters in legal or compliance settings, where absence of recoverable watermark evidence is not the same as evidence of absence.

From feasibility to deployment. The evaluation establishes the channel-knowledge tradeoff: rank-only verification can preserve useful signal when exact-probability verification is unavailable. The strongest current results are cumulative: pooled evidence recovers once enough trajectories or audit-window steps are available, as illustrated by the pooled recovery curve in Fig. 3. A production deployment should add three engineering checks before relying on decoded payloads for attribution: live end-to-end RLNC recovery over the deployed audit window, calibrated cross-model rank reconstruction for the deployed verifier model or API, and pre-registered decision thresholds for positive, inconclusive, and negative outcomes. These checks are operational rather than conceptual, but they are important for turning weak-asymmetric provenance into an audit procedure.

What the contribution is not. AsymMark is not a rebranding of the rank primitive from weakly asymmetric steganography, and AsymMark-R is not a claim that rank-only decoding dominates symmetric decoding on every metric. The primitive is borrowed; the contribution is the agent-watermarking framework around verifier-reconstructible invariants, plus the rank instance, robustness boundary, and audit protocol. AsymMark-R intentionally gives up exact-channel capacity to remove a verifier-channel assumption that is fragile in realistic audits: when calibrated probabilities cannot be trusted but top-k ranks can, decode from the invariant that survives.

8. Related Work

LLM and agent watermarking. Token-level LLM watermarking modifies or tests generated text distributions [4], [5]. Follow-up work studies reliability under paraphrasing, mixed documents, and edit-style perturbations [16], [17]. These methods are complementary to behavioral watermarking because agents often compile natural-language plans into structured actions, where the final text may no longer carry the decision signal. AgentMark [6] is the closest prior work and establishes distribution-preserving behavioral watermarking; AgentMark-F is its exact-probability instance. AsymMark is the weakly asymmetric counterpart: it keeps the policy-preserving objective but makes verifier-channel knowledge an explicit design and evaluation axis rather than assuming the verifier can replay the embedder's probability channel.

Behavior-level provenance for agents. Recent agent watermarking work shows that the behavior trajectory itself can carry provenance evidence. Agent Guide biases an agent toward keyed behavior subsets and detects the resulting trajectory statistic [15]; it is simple and model-agnostic, but its watermark signal comes from an intentional behavior-distribution shift. AgentMark [6] instead preserves the per-step behavior distribution and embeds payload bits through symmetric differential recombination, making it the direct predecessor of this paper. Sequential behavioral watermarking further encodes signals across action transitions and detects them in a position-agnostic way [18]. These lines are complementary to our channel-knowledge question. AsymMark-R is not another detector statistic over selected actions; it asks what channel information the verifier must reconstruct to decode a distribution-preserving behavioral payload. Its answer is top-k rank order rather than exact probability values.

Robustness target. Most text-watermark robustness work asks whether a detector can still identify generated text after rewriting or mixing. Our robustness target is different: the observed behavior may be unchanged, but the verifier's reconstruction of the behavior distribution may differ from the embedder's. This makes probability-value perturbation a channel-knowledge problem rather than an output-editing problem. AsymMark-R addresses this by designing the decoded statistic to be rank position, not a probability-dependent bin.

Steganography and distribution-preserving sampling. Provably secure steganography formalizes indistinguishability between cover and stego content [11]. Meteor provides a practical cryptographic construction for realistic distributions [12], while later work improves payload embedding under language-model channels through undetectable or sparse-sampling constructions [19], [20], [21]. Weakly asymmetric steganography studies receivers with partial channel knowledge and gives the rank-based construction that AsymMark-R relies on [7]. We do not claim this rank primitive as original. The framework-level contribution is to identify verifier-reconstructible invariants as the missing layer for behavioral watermarking; the instance-level contribution is to adapt the rank primitive from token/text channels to planning-behavior channels, couple it with AgentMark's distribution-preserving behavior interface and RLNC recovery layer, and evaluate the resulting verifier-knowledge tradeoff on agent trajectories.

Erasure coding for partial logs. Random linear network coding is a standard way to recover a payload from any sufficiently independent subset of coded symbols [13]. In behavioral watermarking, this abstraction is useful because trajectories are naturally partial: failed steps, missing telemetry, and verifier-side top-k misses all become erasures. Our contribution is not a new erasure code; it is the rank-only step decoder that supplies coded bits to the existing erasure-tolerant layer.

Agent benchmarks. ALFWorld aligns text-based embodied environments with interactive learning [8]. ToolBench evaluates LLM tool-use over real-world APIs [9]. OASIS demonstrates large-scale social-agent simulation [10]. These benchmarks highlight why behavior-level provenance matters: the action trajectory is often more important than the final message.

9. Threats to Validity and Ethics

Offline postprocessing. Capacity and robustness experiments are computed from logged trajectories and offline decoder proxies. This is appropriate for isolating the channel-knowledge question, but it is weaker than a complete live deployment study with end-to-end RLNC recovery under production logging. The paper therefore labels proxy capacity claims explicitly and avoids treating them as full-system recovery guarantees.

Mechanism attribution. The proxy diagnostics show that AgentMark-F loses usable payload signal under rank-only verification even when nominal packet availability remains high, and the added step-level counterfactual attributes a measurable fraction of that loss to probability-bin instability under rank-stable top-k views. This diagnostic still uses logged offline distributions and a top-k truncation counterfactual; it is not a claim that every production verifier will reconstruct the same surrogate distribution.

Verifier perturbations. The robustness package includes controlled rank noise, erasure, empirical false-positive trials, evidence-strength diagnostics, matched top-k reruns, and live cross-model reranking. These studies test the verifier's rank-reconstruction pipeline rather than only the probability-versus-rank decoder boundary isolated in the offline proxy. The coverage is still incomplete: the positive matched-top-k and live rerank claims are ToolBench claims, not a statement that embodied-planning domains such as ALFWorld have equally stable cross-model ranks. The theory covers live reranking only to the extent that the cross-model verifier preserves top-k rank paths; calibrating that preservation remains a deployment requirement.

Benchmark coverage. ALFWorld and ToolBench cover embodied planning and tool-use, but they do not exhaust the space of agentic workflows. Social simulation, GUI control, multi-agent collaboration, and long-running enterprise agents may have different rank stability and trajectory length. The construction is domain-agnostic, but the best k and payload length should be calibrated per environment.

Statistical uncertainty. The evaluation reports aggregate success, JSD, and recovery rates over the completed model/split groups in the result package. These aggregates support the qualitative channel-knowledge comparison, but they should not be read as formal significance tests for small utility differences. A deployment study should pre-register its task distribution, sampling budget, and uncertainty intervals before using small deltas as operational evidence.

Candidate canonicalization. Rank decoding assumes that the verifier can map the logged behavior and reconstructed candidates into the same canonical behavior set. This is straightforward for enumerated actions and structured tool calls, but less automatic for free-form plans, GUI operations, or APIs whose schemas evolve over time. A deployment should canonicalize tool names, arguments, and environment actions before ranking; otherwise a semantically correct but differently serialized behavior may appear outside the verifier's top-k list and become an erasure.

Distribution elicitation. The method assumes that the embedder can elicit or estimate a behavior distribution. If the candidate generator is poor, distribution preservation protects the wrong channel. This is a limitation inherited from behavioral watermarking generally, not a weakness specific to rank-only decoding.

9.1. Ethics Considerations

This work is intended for provenance, audit, and authorized ownership verification. It should not be used to conceal malicious coordination or bypass platform policy. The construction assumes access to a behavior distribution at embedding time and top-k rank information at verification time. It does not protect against arbitrary rank manipulation, model-level watermark removal, or compromised keys. Operational use requires end-to-end RLNC recovery under live logging and cross-model rank reconstruction.

The reported evaluation uses benchmark task trajectories and postprocessed experiment artifacts rather than private user data. A deployment that logs real agent behavior should treat trajectories as potentially sensitive operational records, minimize retained context, and bind verification access to an explicit audit purpose.

Watermarking creates governance risks as well as benefits. A false or overconfident attribution can harm users, developers, or organizations. Deployments should therefore publish the verification threshold, report inconclusive outcomes, retain audit logs, and separate watermark evidence from broader incident investigation. The method should not be used for covert monitoring of users without notice; its appropriate role is provenance for agents or services controlled by the embedding party.

10. Conclusion

Behavioral provenance for LLM agents should not depend on a verifier reproducing exact per-step probabilities. This paper identifies probability-bin instability as a verifier-channel failure mode for symmetric behavioral watermarking and formulates AsymMark, a weakly asymmetric framework that decouples the policy-preserving embedding channel from a verifier-reconstructible decoding invariant. Its rank instance, AsymMark-R, shows that top-k rank order can retain useful watermark signal while preserving the agent's marginal behavior policy. By replacing probability-dependent bins with a rank-decodable binary partition tree, AsymMark-R trades some exact-channel nominal capacity for usable effective capacity under realistic partial-channel verification. This weakly asymmetric view makes behavioral watermarking better aligned with how agents are actually audited: decode from the invariant that survives.

These results establish partial-channel feasibility. Exact-channel AgentMark-F remains attractive when the original behavior probabilities are available; AsymMark targets audit settings where the verifier can reconstruct an invariant but not the full distribution, and AsymMark-R instantiates that idea with ranks. Closing the remaining deployment gap requires longer live trajectories, end-to-end coded-payload recovery, and cross-model rank calibration.

Appendix A. Distribution-Preservation Details

This appendix expands the proof sketch from Section 5. At any internal node of the top-k partition tree, let the even-rank group have normalized mass S_0 and the odd-rank group have mass S_1, with $S_0 + S_1 = S$. Assume without loss of generality that $S_0 \geq S_1$; otherwise the group labels are swapped for the binary primitive. For $r \leftarrow U[0, 1)$ and message bit $m \in \{0, 1\}$, BinEnc computes

$$r_m = (rS + \tfrac{1}{2}Sm) \bmod S.$$

For either fixed m, r_m is uniform over $[0, S)$ because adding a constant modulo S preserves uniformity. Therefore the probability of selecting group j is exactly S_j/S. When the higher-mass group is selected, no bit is consumed; when the lower-mass group is selected, one bit is consumed. Bit consumption changes the message pointer but not the marginal group-selection probability.

Inducting over the recursive partition tree, every visible leaf b is selected with probability equal to its normalized top-n_t mass $P_t(b)/S_{\text{top}}$. The top/tail gate enters the tree with probability $S_{\text{top}} = \sum_{i<n_t} P_t(\sigma_t(i))$ and otherwise samples the tail proportionally. Hence every behavior, including tail behaviors that carry no bits, has marginal probability $P_t(b)$.

Appendix B. DRBG Synchronization

The encoder and decoder derive a per-step HMAC-SHA512 DRBG stream from the shared key and logged context. The important invariant is not the exact digest format, but the fixed consumption budget. For a top-k step with actual visible candidate count $n_t = \min(k, |\mathcal{B}_t|)$, both sides reserve

$$B_t = \lceil \log_2 n_t \rceil$$

DRBG draws for the rank tree. The encoder may terminate early when a singleton leaf is reached or when a zero-bit branch defers the message bit, but it still advances the stream to consume exactly B_t draws. The decoder independently consumes the same B_t draws before returning the recovered bits. The commonly reported $\lceil \log_2 k \rceil$ budget is the special case in which the step fills the requested top-k view. Tail selections emit no bits and are treated as erasures by the verifier.

This fixed-budget rule prevents a local zero-bit event from shifting every later step. Without it, the decoder would have to know whether the encoder consumed a message bit or terminated early at each level, which is precisely the information it is trying to infer from the rank path.

Appendix C. RLNC Payload Recovery

The rank decoder outputs a variable number of coded bits from each observed step. Deterministic RLNC converts these bits into payload equations. For an L-bit payload m, the i-th coded bit is $c_i = \langle a_i, m \rangle \bmod 2$, where a_i is generated from a deterministic coefficient stream. The verifier collects decoded pairs (a_i, c_i) and solves the resulting linear system over GF(2). Recovery succeeds when the collected coefficient matrix has rank L and the decoded right-hand side is consistent.

This layer is agnostic to whether a coded bit came from exact-probability decoding or rank-only decoding. Rank-only verification typically produces fewer reliable equations, so it may need larger audit windows. The capacity diagnostics in the main text use this layer to compare verifier channels; deployment identifiers require longer registered payloads and live end-to-end recovery over pooled logs.

Appendix D. Rank Stability and Top-k Choice

Temperature rescaling preserves rank order because $p_i^{1/T}/Z$ is monotone in p_i for $T > 0$. Quantization, model refreshes, and proxy re-querying need not preserve ranks exactly, but adjacent swaps are most likely when score gaps are small. This motivates top-k restriction: high-probability behaviors usually have larger gaps and more stable semantic alternatives than long-tail candidates.

The top-k parameter therefore trades nominal capacity against reconstruction risk. Larger k gives more leaves and potentially more decoded bits per step, but it asks the verifier to reconstruct lower-probability ranks. Smaller k is conservative and compatible with restricted APIs, but it may underuse high-entropy trajectories. The deployment choice should maximize calibrated $C_{\text{eff}}(k)$, not raw decoded length: a deeper top-k view is useful only when its marginal capacity gain exceeds the marginal decode-success loss. The evaluation reports a sweep rather than selecting a universal k because the best choice is environment-dependent.

Appendix E. Robustness Saturation and Medium-Window Diagnostics

The fully pooled robustness checks are retained as audit sanity checks rather than as primary evidence. They are deliberately conservative and mostly saturated: for DeepSeek top-10 adjacent-swap rank noise, ALFWorld ID and ToolBench all-split pooled recovery remain at 1.0 through $\epsilon = 0.5$, while ALFWorld OOD falls only at the most severe tested point, to 0.83 at $\epsilon = 0.5$. Under fixed step erasure, all reported ALFWorld and ToolBench pooled cells for both AgentMark-F and AsymMark-R recover at 1.0 through 90% erasure. Listing those cells as tables adds little beyond showing that the pooled audit window is overpowered for this diagnostic.

Table 3 instead reports a non-saturated path-consistency view from the fine ToolBench rerun. The metric is not full RLNC payload recovery: it is the probability that a short audit window keeps the decoded rank path unchanged under adjacent swaps. This is closer to the perturbation quantity used in Proposition 2 and exposes the expected decay with noise and window length.

TABLE 3. Medium-window path consistency under adjacent-swap rank noise. ToolBench, DeepSeek, G1 instruction split, 500 trials per cell. Values are path-window success rates, not full payload recovery.

	$\epsilon = 0.05$			$\epsilon = 0.25$			$\epsilon = 0.50$		
k	4-step	8-step	12-step	4-step	8-step	12-step	4-step	8-step	12-step
3	.794	.656	.504	.498	.198	.090	.174	.034	.004
5	.728	.572	.406	.182	.042	.016	.064	.002	.000
10	.704	.502	.388	.246	.056	.016	.046	.000	.000

Appendix F. Vanilla Baseline

The vanilla ALFWorld result is reported only as a task-performance sanity check for the explicit prompting and clean AgentMark pipeline used by the main comparisons. On the canonical 0510 ALFWorld artifact, the clean induced pipeline reaches 80.0% success with 16.23 steps, while the vanilla agent reaches 69.1% success with 16.54 steps. Table 4 expands this aggregate by model and split. This comparison is not a watermarking baseline; it shows that the explicit behavior-induction interface used to expose candidate actions does not degrade ALFWorld task performance before watermarking is applied.

TABLE 4. ALFWORLD VANILLA SANITY CHECK FOR EXPLICIT BEHAVIOR INDUCTION, EXPANDED BY MODEL AND SPLIT. SUCCESS AND STEPS ARE MEAN±STANDARD DEVIATION OVER THE COMPLETED RUNS.

Method	Model	Split	Runs	Success (%)	Steps
Vanilla	Gemini	ID	3/3	60.95±1.09	18.45±0.07
Vanilla	Gemini	OOD	3/3	66.67±1.55	17.67±0.49
Vanilla	DeepSeek	ID	3/3	71.67±1.65	15.59±0.56
Vanilla	DeepSeek	OOD	3/3	77.11±0.86	14.46±0.39
Clean induced	Gemini	ID	3/3	72.62±0.82	17.34±0.08
Clean induced	Gemini	OOD	3/3	69.40±0.00	18.76±0.00
Clean induced	DeepSeek	ID	3/3	88.57±0.00	14.64±0.01
Clean induced	DeepSeek	OOD	3/3	89.55±0.00	14.19±0.03

Appendix G. Reproducibility Mapping

The reported tables are tied to postprocessed artifacts at the claim level:

- Utility and stealth claims are backed by task-success, step-count, and behavior-frequency summaries over the completed benchmark groups.
- Verifier-channel claims are backed by exact, rank-only, and selected-only decoder views over the same logged trajectory corpus.
- Mechanism claims are backed by a probability-bin counterfactual that holds rank order fixed while perturbing probability values.
- Top-k calibration claims are backed by offline rank-depth sweeps and live cross-model reranking studies.
- Robustness claims are backed by rank-noise and erasure studies using pooled audit-window recovery.
- Evidence claims are backed by random-claim false-positive trials and bit-level confidence diagnostics.

Machine-readable sidecars mirror these summaries for automated checks.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[2] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[3] Gemini Team, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[4] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, "A watermark for large language models," in *Proceedings of the International Conference on Machine Learning*, 2023.

[5] S. Dathathri, A. See, S. Ghaisas, P.-S. Huang, R. McAdam, J. Welbl, V. Bachani, A. Kaskasoli, R. Stanforth, T. Matejovicova, J. Hayes, N. Vyas, T. Zacchary, M. Zilka, I. Gabriel, W. Isaac, A. Guez, L. Weidinger, J. Mellor, D. Hassabis, K. Kavukcuoglu, G. Irving, and P. Kohli, "Scalable watermarking for identifying large language model outputs," *Nature*, 2024.

[6] K. Huang, J. Tan, Y. Wei, W. Li, Z. Zhang, H. Tian, Z. Yang, and L. Zhou, "AgentMark: Utility-preserving behavioral watermarking for agents," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2026.

[7] Anonymous authors, "Title omitted for double-blind review," Anonymous manuscript, 2026.

[8] M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht, "ALFWorld: Aligning text and embodied environments for interactive learning," in *Proceedings of the International Conference on Learning Representations*, 2021.

[9] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world apis," *arXiv preprint arXiv:2307.16789*, 2023.

[10] C. Gao, X. Lan, Z. Lu, J. Mao, J. Piao, H. Wang, D. Jin, and Y. Li, "OASIS: Open agents social interaction simulations with one million agents," *arXiv preprint arXiv:2411.11581*, 2024.

[11] N. J. Hopper, J. Langford, and L. von Ahn, "Provably secure steganography," in *Proceedings of the Annual International Cryptology Conference*, 2002.

[12] G. Kaptchuk, T. M. Jois, M. Green, and A. D. Rubin, "Meteor: Cryptographically secure steganography for realistic distributions," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2021.

[13] T. Ho, M. Médard, R. Koetter, D. R. Karger, M. Effros, J. Shi, and B. Leong, "A random linear network coding approach to multicast," *IEEE Transactions on Information Theory*, vol. 52, no. 10, pp. 4413–4430, 2006.

[14] DeepSeek-AI, "DeepSeek llm: Scaling open-source language models with longtermism," *arXiv preprint arXiv:2401.02954*, 2024.

[15] K. Huang, Z. Zhang, Z. Yang, and L. Zhou, "Agent guide: A simple agent behavioral watermarking framework," *arXiv preprint arXiv:2504.05871*, 2025.

[16] J. Kirchenbauer, J. Geiping, Y. Wen, M. Shu, K. Saifullah, K. Kong, K. Fernando, A. Saha, M. Goldblum, and T. Goldstein, "On the reliability of watermarks for large language models," in *Proceedings of the International Conference on Learning Representations*, 2024.

[17] R. Kuditipudi, J. Thickstun, T. Hashimoto, and P. Liang, "Robust distortion-free watermarks for language models," *Transactions on Machine Learning Research*, 2024.

[18] H. An, S. Park, D. Kim, and Y.-S. Han, "Sequential behavioral watermarking for LLM agents," *arXiv preprint arXiv:2605.11036*, 2026.

[19] M. Christ, S. Gunn, and O. Zamir, "Undetectable watermarks for language models," in *Proceedings of the Conference on Learning Theory*, 2024.

[20] O. Zamir, "Undetectable steganography for language models," *Transactions on Machine Learning Research*, 2024.

[21] Y. Wang, G. Pei, K. Chen, J. Ding, C. Pan, W. Pang, D. Hu, and W. Zhang, "SparSamp: Efficient provably secure steganography based on sparse sampling," in *Proceedings of the 34th USENIX Security Symposium*, 2025.